

13.3 FSM Using Verilog Behavioral Modeling

Figure 13.1 shows the standard approach to modeling at a fairly low level. Because modeling FSMs at a low level is not efficient, we first want to use a BBD to describe how will model FSMs at a higher level of abstraction. Figure 13.2 shows a BBD modeling our Verilog behavior descriptions of FSM; here are a few items of interest to note about this model.

- There are two blocks: a sequential block and a combinatorial block. This means that we now divide the functionality of the three boxes in Figure 13.1 into the two boxes in Figure 13.2.
- The SEQ_CKT box is the sequential circuit that handles the state registers. This box is synchronous but it often has asynchronous control inputs such as reset signals. The main purpose of the SEQ_CKT box is to implement the state changes in the circuit; thus the COMB_CKT box determines what the next state should be based on the present state and external inputs, the SEQ_CKT latches that new state into the state registers thus making it the present state.
- The COMB_CKT box is a combinatorial circuit that essentially implements both the next state and output decoder. The next state is a function of the present state and the external inputs. The COMB_CKT box also determines the values for the Mealy and Moore-type outputs.

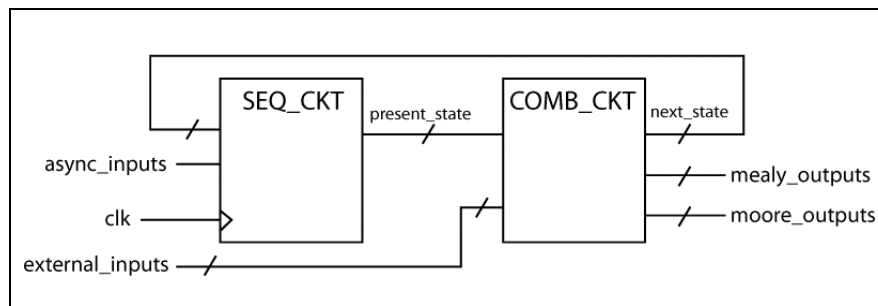
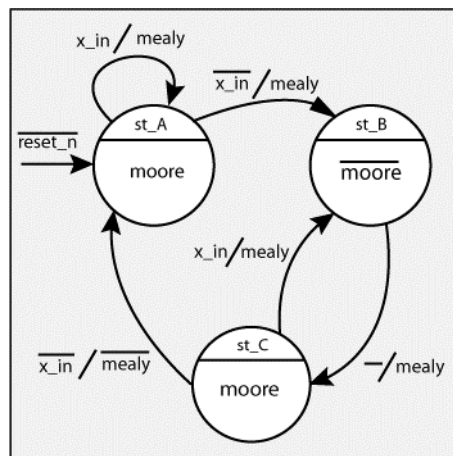


Figure 13.2: A BBD showing the Verilog behavioral modeling approach to implementing FSMs.

Example 13.1: Verilog FSM Model Implementation #1

Use a Verilog behavioral model to describe the follow state diagram.



Solution: This is a contrived state diagram that shows how to implement every possible characteristic in a FSM. Note that from the state diagram we have both a Mealy and Moore-type output, an asynchronous reset input, and one external input. It does not get any more complicated than this. Figure 13.3 show the results of

the first step in this example, which is to generate a BBD. Figure 13.4 shows the final solution to this example. Here are all the features to note about this example.

- The modeling approach uses two **always** blocks, one block is for the combinatorial processes that handles issues specific to the output and next state decoder; the other **always** block handles issues associated with the state registers, including asynchronous inputs.
- The model must first create variables that handle the state mechanism, as well as how exactly the FSM encodes those states. The model gives the appropriate names of **NS** and **PS** for the state information, which not surprisingly stands for next state and present state. The model then uses the parameter data-type to assigned specific binary values to each of the states in the state diagram. The **parameter** keyword assigns constant values to the state names.
- The state diagram has three states, so we need to use at least two bits as state variables. We arbitrarily chose the different bit combinations, which leave ones bit combination used, which is potentially a hang state.
- We prefix the state names with a “st_” to give the human reader that the particular variable is the name of a state, which represents good self-commenting Verilog code.
- The **always** block associated with the sequential process is sensitive to the edges: the negative edge of the **reset_n** signal and the positive edge of the **clk** signal. The **reset_n** signal is asynchronous because its evaluation is independent of the **clk** signal. The **reset_n** signal also has a higher precedence than the **clk** signal based on the sequential¹⁶ nature of the code in the body of the **always** block. All signal assignments in the **always** block are blocking.
- The second **always** block (the sequential **always** block) is sensitive to changes in the external input signal (**x_in**) and changes in the signal representing the present state (**PS**). Thus, a change in either of these signals causes the process to be evaluated.
- The first thing we do in the second **always** block is to assign all external outputs a value, which we do to ensure the synthesizer never generates a latch for any output.
- The sequential **always** block contains one procedural programming statement: a case statement. This **case** statement has one clause for each state in the state diagram and one default clause to make the FSM self-correcting.
- Each clause in the **case** statement has three responsibilities: 1) assign the Moore-type outputs, 2) assign the Mealy-type outputs, and, 3) assign the next state. The Moore-type output assignments are a function of the state only, so that assignment always appears first in the associated state. Both the next state (**NS**) and Mealy-type outputs are a function of the external inputs, so we must use a procedural programming statement (an if-else) to correctly assign both **NS** and the Mealy-type output.
- The “st_B” state transitions unconditionally to the “st_C” state. Because the external input does not control this transition, the Mealy-type output is constant. We thus have no **if-else** clause in this section of the case statement.
- All the **case** clauses use **if-else** procedural programming statements, meaning that using the **else** clause provides a catch-all statement to partially ensure the synthesizer generates no latches and to ensure the FSM is deterministic.
- The case statement contains a default statement. The FSM should never find itself in this condition, but if it does, the FSM uses the default external output assignments and directs the FSM to state “st_A”.

¹⁶ This means sequential as the **always** block reads the statements in the order they appear; this does not mean sequential in relation to a type of digital circuit.

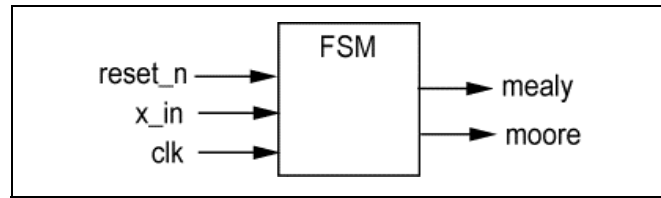


Figure 13.3: The BBD for this problem's FSM.

```

module fsm_template(reset_n, x_in, clk, mealy, moore);
  input  reset_n, x_in, clk;
  output reg mealy, moore;

  //- next state & present state variables
  reg [1:0] NS, PS;
  //- bit-level state representations
  parameter [1:0] st_A=2'b00, st_B=2'b01, st_C=2'b11;

  //- model the state registers
  always @ (negedge reset_n, posedge clk)
    if (reset_n == 0)
      PS <= st_A;
    else
      PS <= NS;

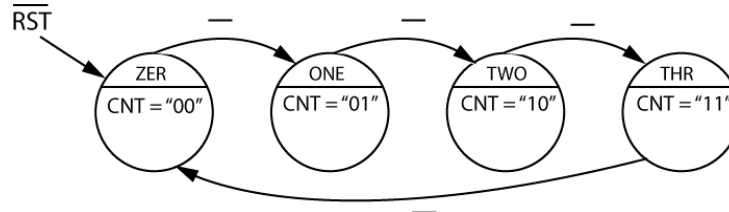
  //- model the next-state and output decoders
  always @ (x_in,PS)
  begin
    mealy = 0; moore = 0; // assign all outputs
    case(PS)
      st_A: //-----
      begin
        moore = 1;
        if (x_in == 1)
          begin
            mealy = 0;
            NS = st_A;
          end
        else
          begin
            mealy = 1;
            NS = st_B;
          end
        end
      st_B: //-----
      begin
        moore = 0;
        mealy = 1;
        NS = st_C;
      end
      st_C: //-----
      begin
        moore = 1;
        if (x_in == 1)
          begin
            mealy = 1;
            NS = st_B;
          end
        else
          begin
            mealy = 0;
            NS = st_A;
          end
        end
      default: NS = st_A;
    endcase
  end
endmodule

```

Figure 13.4: The solution to Example 13.1.

Example 13.2: Verilog FSM Model Implementation #1

Use a Verilog behavioral model to describe the follow state diagram.



Solution: The state diagram provides all the information we need to complete this problem. Figure 13.5 shows the result of the best first step to take, which is to draw the BBD for the FSM. You certainly don't have to draw the BBD, but doing so helps prevent dumbstarted mistakes in the Verilog modeling process. We can note from the state diagram that this FSM has one output: **CNT**, which is a Moore-type output. All state transitions are unconditional, and the circuit has an active low reset input (**RST**), which places the FSM into the ZER state.

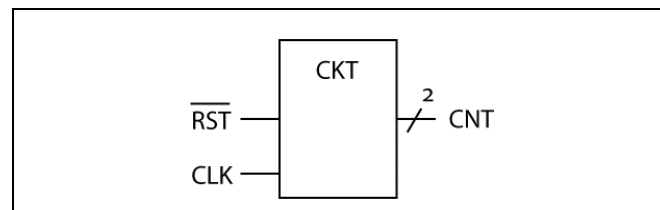


Figure 13.5: The BBD for this problem's FSM.

Figure 13.6 shows the final Verilog model for this example; here are a few non-boring things to note about this solution.

The code in the model uses white space (blank line & indentation) to enhance readability. The code is adequately commented as well.

All state transitions are unconditional, so the individual clauses in the case statement do not include **if-else** constructs.

The state names in the model closely match the state labels in the state diagram, which enhances the understandability of the model to human viewers.

```
module fsm_example_01(rst, clk, cnt);
    input rst, clk;
    output reg [1:0] cnt;

    //- next state & present state variables
    reg [1:0] NS, PS;
    parameter [1:0] st_ZER=2'b00, st_ONE=2'b01, st_TWO=2'b10, st_THR=2'b11;

    //- model the state registers
    always @ (negedge rst, posedge clk)
        if (rst == 0)
            PS <= st_ZER;
        else
            PS <= NS;

    //- model the next-state and output decoders
    always @ (PS)
    begin
        case(PS)
            st_ZER: //------
            begin
                cnt = 2'b00;
                NS = st_ONE;
            end

            st_ONE: //------
            begin
                cnt = 2'b01;
                NS = st_TWO;
            end

            st_TWO: //------
            begin
                cnt = 2'b10;
                NS = st_THR;
            end

            st_THR: //------
            begin
                cnt = 2'b11;
                NS = st_ZER;
            end

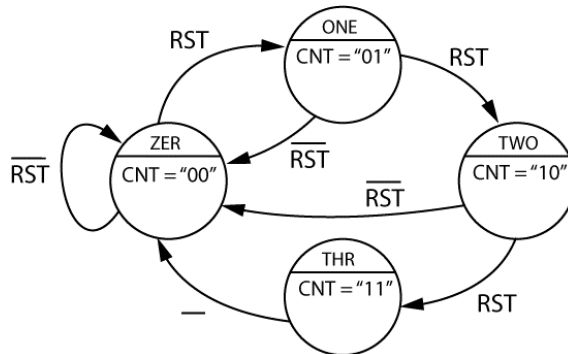
            default: NS = st_ONE;

        endcase
    end    //- always
endmodule
```

Figure 13.6: The solution to Example 13.2.

Example 13.3: Verilog FSM Model Implementation #3

Use a Verilog behavioral model to describe the follow state diagram.



Solution: This problem is the similar to the previous problem, but now the **RST** input is synchronous. The first thing to note is that the **RST** is always associated with state-to-state transitions, rather than the nowhere-to-state transitions associated with the asynchronous flavor of **RST**. Figure 13.7 shows the BBD associated with this FSM while Figure 13.8 shows the final solution.

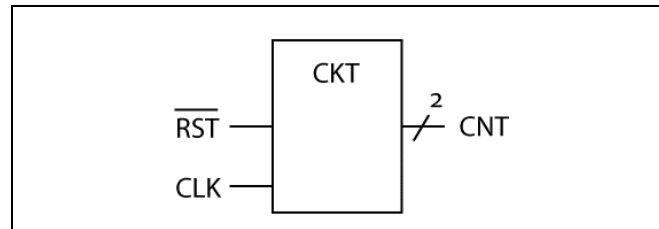


Figure 13.7: The BBD for this problem's FSM.

The major differences between the solution to this problem and the solution to the previous problem involve the modeling of the **RST** input. Here are some specifics:

- The model using the asynchronous **RST** signal handles the signal in the sequential process while the asynchronous version handles **RST** in the combinatorial process. We could have modeled **RST** in the combinatorial process, but it avoids later confusion to reserve asynchronous signal implementation for the sequential process.
- Because we now model **RST** in the combinatorial process, each of the clauses of the **case** statement now contain **if-else** statements. The associated always block is combinatorial, so we must include an **else** statement.
- In state “st_THR”, the **RST** signal does not matter as the FSM always transitions to “st_ZER”. Thus, the state transitions are a function of **RST** and the state for the all states but “st_THR”, where it is a function of state only.
- The **CNT** output is only a function of state for we model it as a Moore-type output.
- The model contains a default clause. This FSM uses all the 2-bit code space, so illegal state recovery is not an issue for this model.

```
module fsm_example_02(rst, clk, cnt);
    input rst, clk;
    output reg [1:0] cnt;

    //- next state & present state variables
    reg [1:0] NS, PS;
    parameter [1:0] st_ZER=2'b00, st_ONE=2'b01, st_TWO=2'b10, st_THR=2'b11;

    //- model the state registers
    always @ (posedge clk)
        if (clk == 1)
            PS <= NS;

    //- model the next-state and output decoders
    always @ (PS)
    begin
        case(PS)
            st_ZER: //-----
            begin
                cnt = 2'b00;
                if (rst == 0)
                    NS = st_ZER;
                else
                    NS = st_ONE;
            end

            st_ONE: //-----
            begin
                cnt = 2'b01;
                if (rst == 0)
                    NS = st_ZER;
                else
                    NS = st_TWO;
            end

            st_TWO: //-----
            begin
                cnt = 2'b10;
                if (rst == 0)
                    NS = st_ZER;
                else
                    NS = st_THR;
            end

            st_THR: //-----
            begin
                cnt = 2'b11;
                NS = st_ZER;
            end

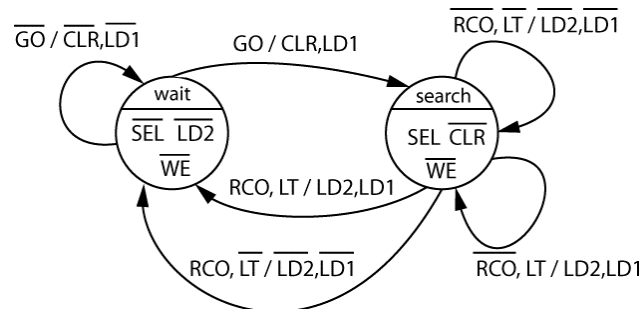
            default: NS = st_ZER;

        endcase
    end
endmodule
```

Figure 13.8: The solution to Example 13.3.

Example 13.4: Verilog FSM Model Implementation #4

Use a Verilog behavioral model to describe the follow state diagram.



Solution: This state diagram has fewer states than the previous examples, but the state transitions and associated conditions are more complicated. The state diagram indicates that the FSM has four inputs: **RCO**, **GO**, **LT**, and **CLK** (implied). The state diagram indicates that the FSM will have five outputs: **LD1**, **LD2**, **SEL**, **WE**, and **CLR**. We know that **WE** and **SEL** are Moore-type outputs because they are only a function of the state; the other outputs are Mealy-type outputs because they are a function of both state and external inputs. Figure 13.9 shows the result of the first step in this solution, which is to draw the BBD.

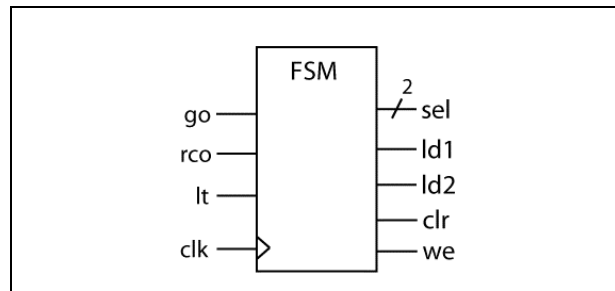


Figure 13.9: The top-level BBD for the FSM.

The next step in this solution is to generate the Verilog model; Figure 13.10 shows the solution. Here are a few things to note about this solution.

- A state diagram of this complexity lends itself of an infinite number of models. The one we show in Figure 13.10 is the one we feel is more appropriate based on space limitations of this text and complexity of the model in general. This is not necessarily the optimal solution, but it fits on a page without shrinking the font too small.
- The model uses a combination of **if-else** clauses both with and without **begin-end** pairs. Once again, we do this for space considerations and not for readability.
- The second clause in the **case** statement could have used an embedded case statement, but we opted to use if-else clauses. We could have been clever here and assigned everything in one statement, but we opted to keep the solution as clear and simple as possible.
- For both if-else clauses in the “st_search” state, we both list every possible input combination and also provide an **else** clause. We could have used one less **if-else** clause and replaced it with an **else**, but we always like to see all possible options listed as part of the **if-else** clause rather than relying on **else** clause to cover the final set of output options.

```

module fsm_example_03(go, rco, lt, clk, we, ld1, ld2, clr, sel);
  input go, rco, lt, clk;
  output reg clr, ld1, ld2, we, sel;

  //- next state & present state variables
  reg NS, PS;
  parameter st_wait=1'b0, st_search=1'b1;

  //- model the state registers
  always @ (posedge clk)
    if (clk == 1)
      PS <= NS;

  //- model the next-state and output decoders
  always @ (PS)
  begin
    we = 0; ld1 = 0; ld2 = 0; clr = 0; sel = 0;
    case(PS)
      st_wait: //-----
      begin
        sel = 0; we = 0; ld2 = 0;
        if (go == 0) begin
          clr = 0; ld1 = 0; end
        else
          begin
            clr = 1; ld1 = 1;
          end
        end
      end

      st_search: //-----
      begin
        //- handle outputs
        sel <= 0; we <= 0; clr <= 0;
        if (rco == 0 && lt == 0) begin
          ld2 <= 0; ld1 <= 0; end
        else if (rco == 0 && lt == 1) begin
          ld2 <= 1; ld1 <= 1; end
        else if (rco == 1 && lt == 0) begin
          ld2 <= 1; ld1 <= 1; end
        else if (rco == 1 && lt == 1) begin
          ld2 <= 0; ld1 <= 1; end
        else
          begin
            ld2 <= 0; ld1 <= 1;
          end
        end

        //- handle state transitions
        if (rco == 1)
          NS <= st_wait;
        else if (rco == 0)
          NS <= st_search;
        else
          NS <= st_wait;
        end

        default: NS = st_wait;
      endcase
    end
  endmodule

```

Figure 13.10: The solution to Example 13.4.

13.4 Chapter Summary

- Many different technical fields use FSMs to model the operations of processes; digital design primarily uses FSMs as digital circuits that control other digital circuits.
 - The engineering step in FSM design is the creation of the state diagram; most everything else beyond that point is grunt-work. The state diagram indicates how it controls a circuit; any type of model of the state diagram is much less impressive.
 - The accepted FSM hardware model includes three basic parts: 1) next state decoder, 2) output decoder, and 3) state registers, where only the state registers are a sequential circuit. There are several approaches to modeling a FSM using Verilog, the most simple model uses two blocks: one to handle the combinatorial sections of the FSM (the next state and output decoders), and one to handle the sequential part of the FSM (the state registers).
 - Modeling FSMs using Verilog is a cut & paste process based on the notion that the challenging step is generating the original state diagram. Working from a template is always the best approach rather than starting from scratch.
-